Manual de entrega - API REST NestJS, MySQL y RabbitMQ

Laboratorio: Optativa IV - Sistemas de Tiempo Real Distribuidos

Fundación Escuela Tecnológica de Neiva (FET)

Taller 2: Mensajería con RabbitMQ en contenedores Docker

1. Contenido del paquete de entrega

El ZIP contiene el código fuente (sin node_modules), Docker, el script SQL, las colecciones de Postman, las capturas y este manual. El PDF está en la raíz de la carpeta de entrega y también en API_NEST/docs/.

- Código fuente: carpeta API_NEST/
- Imagen y composición: Dockerfile y docker-compose.yml
- Script SQL de clientes: sql/01_create_database_and_clients.sql
- Script SQL de pagos: sql/02_create_pagos_and_procesamientos.sql
- Postman de pagos: postman/Pagos_RabbitMQ.postman_collection.json
- Postman de clientes: postman/API_Nest_Clients.postman_collection.json
- Postman de mensajería: postman/Mensajeria_RabbitMQ.postman_collection.json
- Capturas: postman/*.png y postman/*.jpeg
- Este manual: Manual_Entrega_API_Nest.pdf
- Informe del taller 4 (APM y estrés): Informe_Taller_04_APM.pdf y docs/Informe_Taller_04_APM.pdf

2. Repositorio

El código también está publicado en:

```
https://github.com/SamirCortes/API_NEST.git
```

```
git clone https://github.com/SamirCortes/API_NEST.git
cd API_NEST
```

3. Cómo levantar todo con un solo comando

En el equipo solo hace falta Docker. No se instala Node, RabbitMQ ni MySQL. Con Docker encendido:

```
cd API_NEST
docker compose up -d --build
```

Ese comando construye la imagen y levanta MySQL, RabbitMQ, la API productora y el consumidor. La cola messages_queue es durable y se crea sola al conectar.

- API productora: http://localhost:3020
- Swagger: http://localhost:3020/api/docs
- MySQL en el equipo: localhost:3307 (usuario nest, clave nest)
- RabbitMQ AMQP: localhost:5672
- Consola RabbitMQ: http://localhost:15672 (usuario nest, clave nest)

Dentro de la red de Docker los servicios se llaman mysql y rabbitmq, no localhost. La API publica el puerto 3020 porque en este equipo el 3000 estaba ocupado. Dentro del contenedor el proceso escucha en el 3000.

```
docker compose logs -f consumer
docker compose logs -f api
docker compose down
```

4. Productor

El contenedor de la API expone POST /messages. Recibe un JSON, lo publica en messages_queue y responde Content-Type: application/json. El campo status es booleano.

```
{"sensor": "OD-01", "valor": 4.2, "unidad": "mg/L"}
```

Si el mensaje se publica:

```
{"status": true, "message": "Mensaje encolado"}
```

Si el cuerpo no es JSON válido:

```
{"status": false, "message": "Formato de mensaje inválido"}
```

Crear, actualizar o borrar un cliente también publica un evento en la misma cola. Consultar o listar no publica nada.

5. Consumidor

El contenedor api_nest_consumer usa la misma imagen y ejecuta node dist/consumer.js. No abre un puerto. Se conecta a messages_queue, escribe en su log el contenido y la hora, y solo después confirma el mensaje para retirarlo de la cola. Si RabbitMQ aún no está listo, reintenta la conexión.

```
docker compose logs -f consumer
```

6. Casos que se comprueban

1. Consumidor detenido

```
docker compose stop consumer
# Publicar varios POST /messages
# En la consola, messages_queue muestra Ready > 0
docker compose start consumer
docker compose logs -f consumer
```

Los mensajes se acumulan y, al levantar el consumidor, se procesan todos. La cola vuelve a Ready 0.

2. Reinicio de RabbitMQ

Con el consumidor detenido y mensajes en Ready se ejecuta docker compose restart rabbitmq. La cola es durable y los mensajes son persistentes, así que siguen en la cola al volver la consola. En la captura de los 5 mensajes se ve durable: true y Persistent: 5.

```
docker compose restart rabbitmq
```

3. Mensaje inválido

```
curl -i -X POST http://localhost:3020/messages \
  -H "Content-Type: application/json" \
  -d '{sensor: OD-01}'
```

La respuesta lleva status en false y el contador de la cola no aumenta.

7. Registro de pagos

POST /pagos valida los datos, inserta el pago con estado REGISTRADO, publica solo su id en messages_queue y responde enseguida. GET /pagos/:id consulta ese pago. El consumidor espera PROCESS_DELAY_MS, pasa el estado a PROCESADO, guarda un comprobante en procesamientos y solo entonces confirma el mensaje.

```
POST http://localhost:3020/pagos
{"referencia": "PAG-0001", "valor": 125000, "medio": "transferencia"}

{"status": true, "message": "Pago registrado", "data": {"id": 1, "estado": "REGISTRADO"}}
```

Datos inválidos, sin insertar ni publicar:

```
{"status": false, "message": "Datos del pago inválidos"}
```

En el caso 1 el retardo era de 4 segundos. En el log, el pago 3 se tomó a las 23:03:41.564Z y terminó a las 23:03:45.598Z. El pago 7 terminó a las 23:04:01.677Z. La API ya había respondido REGISTRADO antes de eso.

En el caso 2, PROCESS_DELAY_MS pasó a 15000. El pago 16 se registró a las 23:12:21.692Z y la API respondió REGISTRADO en esa misma petición. El consumidor lo tomó a las 23:12:21.705Z y lo finalizó a las 23:12:36.759Z, unos 15 segundos después. El tiempo de respuesta de la API no siguió esa espera.

En el caso 3, la referencia PAG-FALLA hace fallar el procesamiento. El pago 2 permanece en REGISTRADO, no tiene fila en procesamientos y el log dice que el mensaje vuelve a la cola.

```
docker compose logs -f consumer
# Caso 2: PROCESS_DELAY_MS=15000 y docker compose up -d consumer
```

8. Colecciones de Postman

Mensajería

```
postman/Mensajeria_RabbitMQ.postman_collection.json
```

Importar en Postman. La variable baseUrl es http://localhost:3020. La petición 1 envía un mensaje válido y la 2 un cuerpo que no es JSON. Para diez mensajes válidos, usar el Collection Runner solo con la petición 1 y 10 iteraciones.

Pagos

```
postman/Pagos_RabbitMQ.postman_collection.json
```

Tiene tres peticiones: pago válido, pago inválido y consulta por id. baseUrl es http://localhost:3020. La consulta hay que ejecutarla al registrar y otra vez unos diez segundos después.

Clientes

```
postman/API_Nest_Clients.postman_collection.json
```

Ejecutar las peticiones del CRUD en orden. baseUrl también apunta al puerto 3020.

9. API de clientes

- POST /clients crea un cliente y publica client.created
- GET /clients y GET /clients/:id solo consultan
- PUT /clients/:id actualiza y publica client.updated
- DELETE /clients/:id elimina y publica client.deleted

La tabla clients se crea con la migración al arrancar la API, o con sql/01_create_database_and_clients.sql. Las tablas pagos y procesamientos están en sql/02_create_pagos_and_procesamientos.sql.

10. Declaración de uso de herramientas de inteligencia artificial

Se utilizó Cursor, con el modelo Grok, como asistencia durante el laboratorio.

Para qué se usó:

- Comparar el taller de RabbitMQ con la API de clientes ya existente
- Separar el productor y el consumidor en contenedores distintos
- Publicar mensajes persistentes en la cola durable messages_queue
- Ajustar POST /messages y POST /pagos al formato de respuesta pedido
- Agregar las tablas pagos y procesamientos y el procesamiento asíncrono
- Actualizar Docker Compose, el README, Swagger y las colecciones de Postman
- Generar este manual y el paquete ZIP de entrega
- Instrumentar la API con OpenTelemetry, desplegar SigNoz y escribir el escenario k6 del taller 4
- Tomar las capturas de SigNoz y redactar el informe con la tabla, las gráficas y el análisis de la corrida del 26 de octubre de 2026

Qué se revisó o hizo sobre ese resultado:

- Pruebas manuales en Swagger, Postman, DBeaver y la consola de RabbitMQ

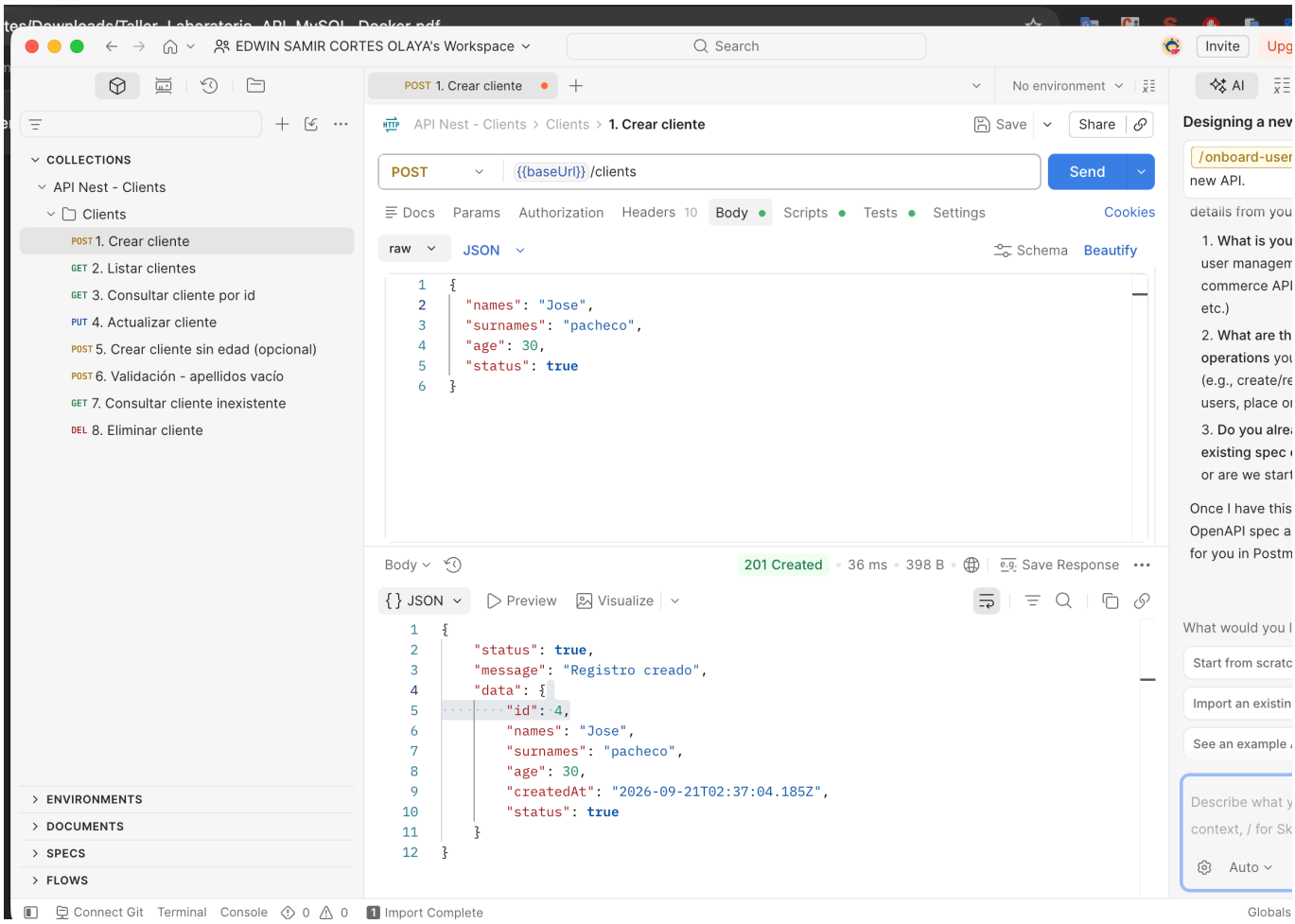
- Capturas de los casos del taller 3: consumidor detenido, pausa de 15 segundos y fallo PAG-FALLA
- El puerto publicado de la API quedó en 3020 porque el 3000 estaba ocupado

El estudiante asume la responsabilidad del código entregado, de su funcionamiento y de la comprensión de la solución.

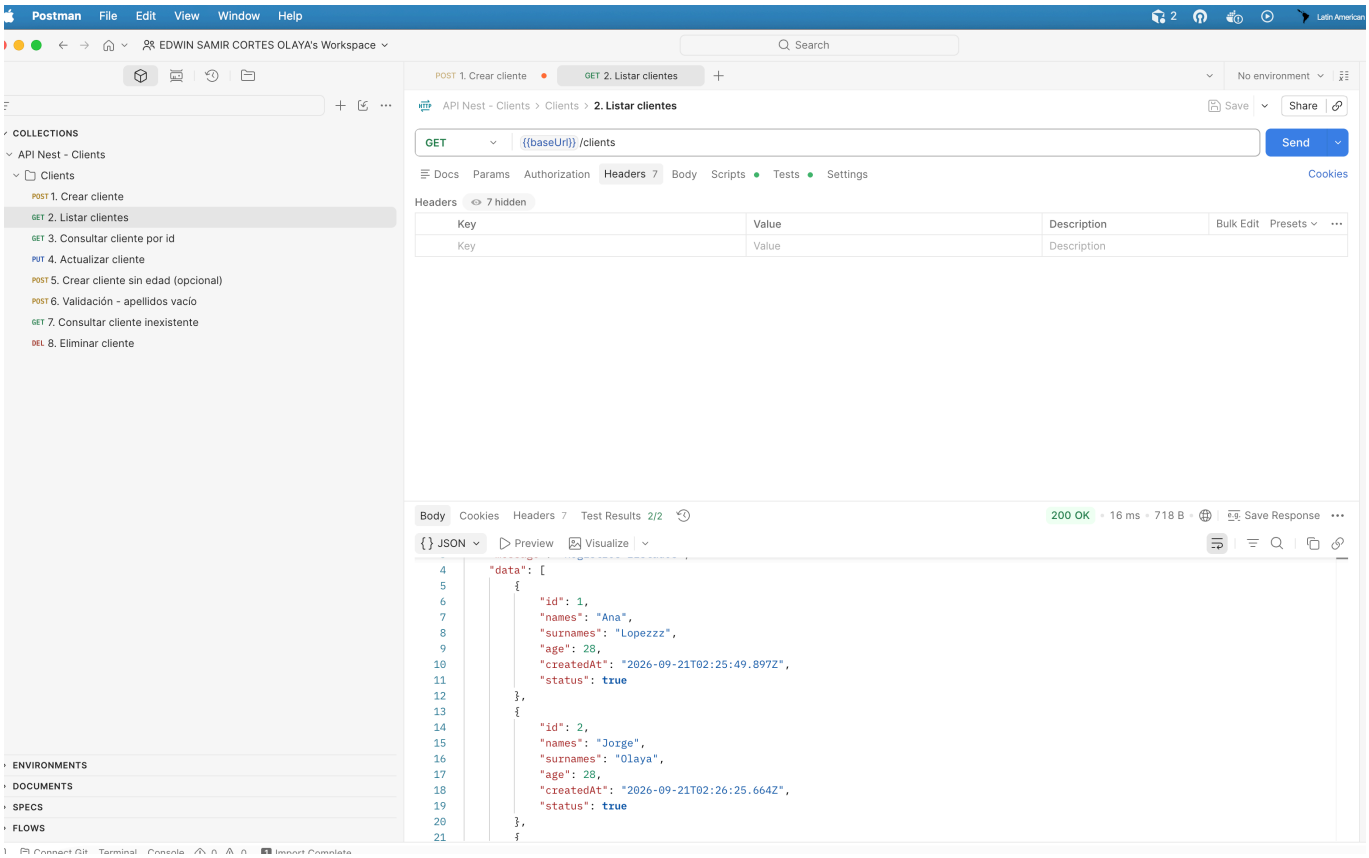
11. Evidencias del CRUD de clientes

Capturas de Postman de las peticiones del laboratorio anterior.

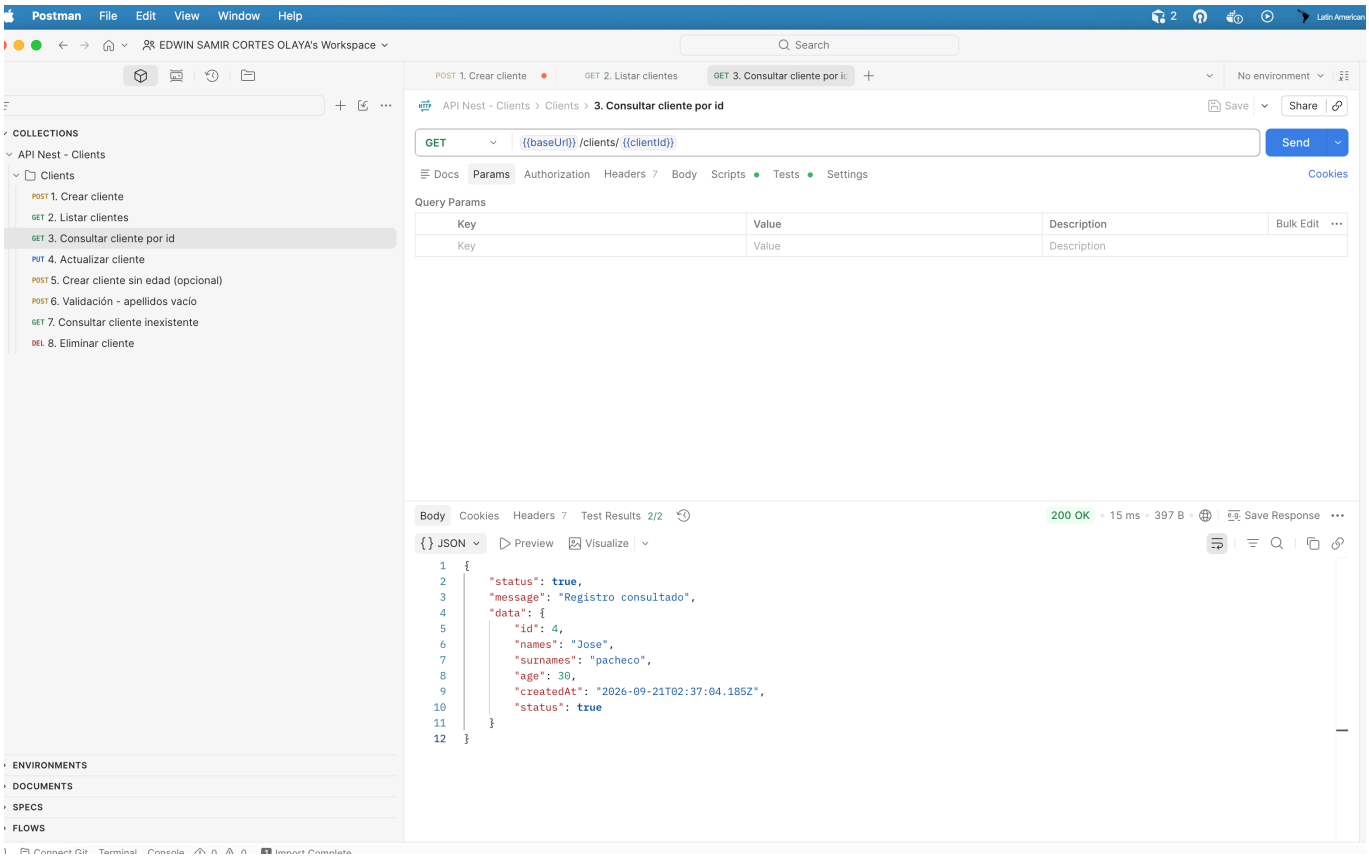
1. Crear cliente (POST /clients)



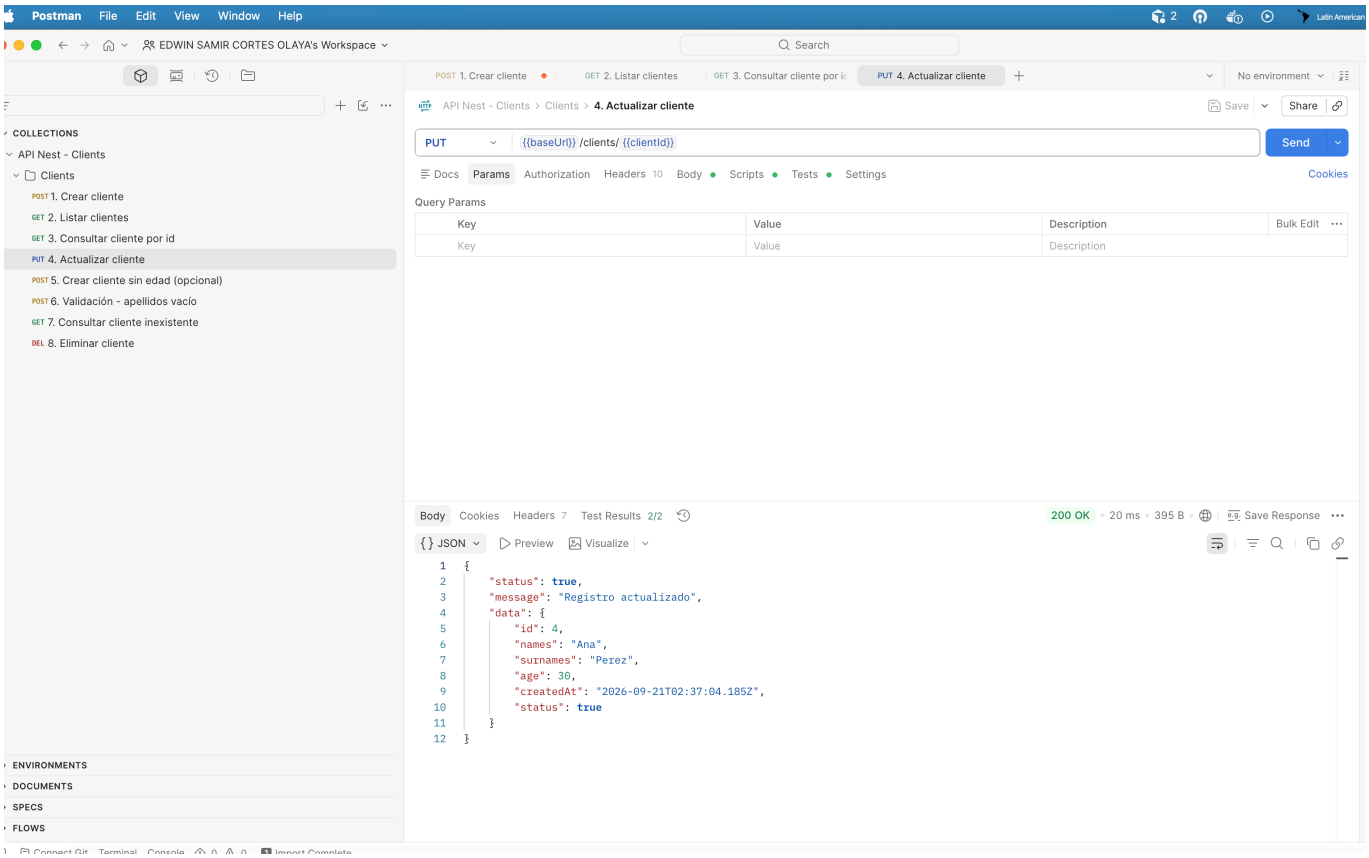
2. Listar clientes (GET /clients)



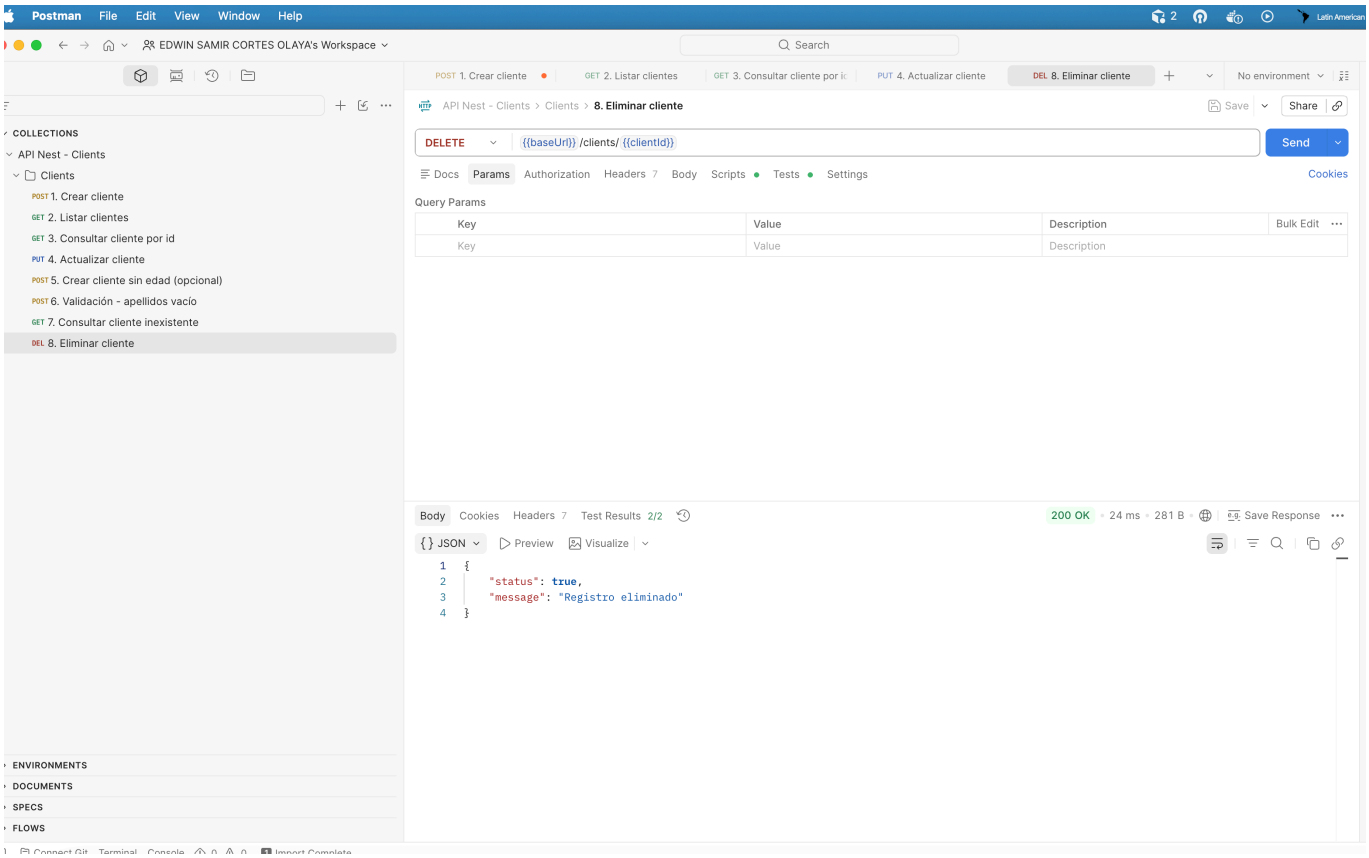
3. Consultar cliente por id (GET /clients/:id)



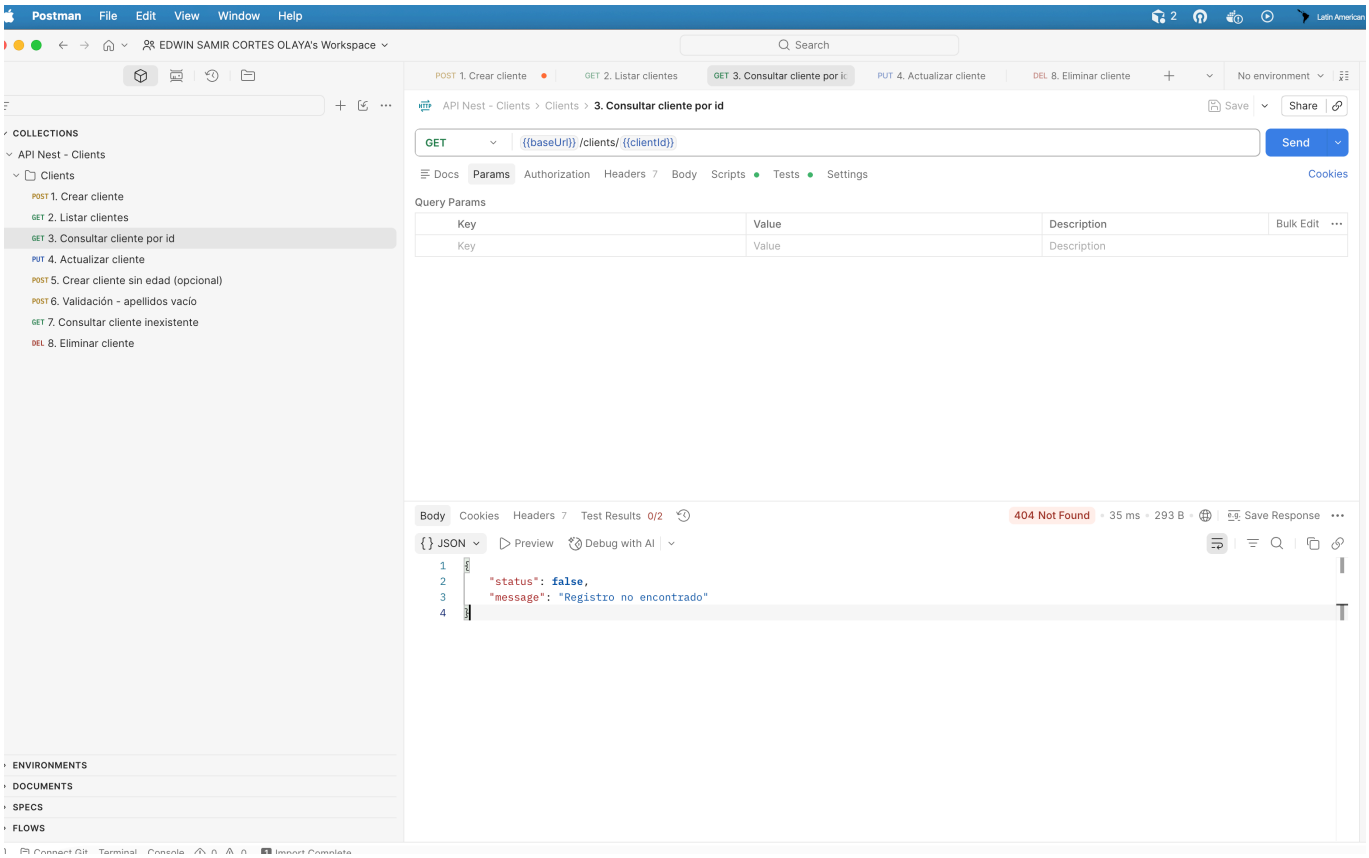
4. Actualizar cliente (PUT /clients/:id)



5. Eliminar cliente (DELETE /clients/:id)



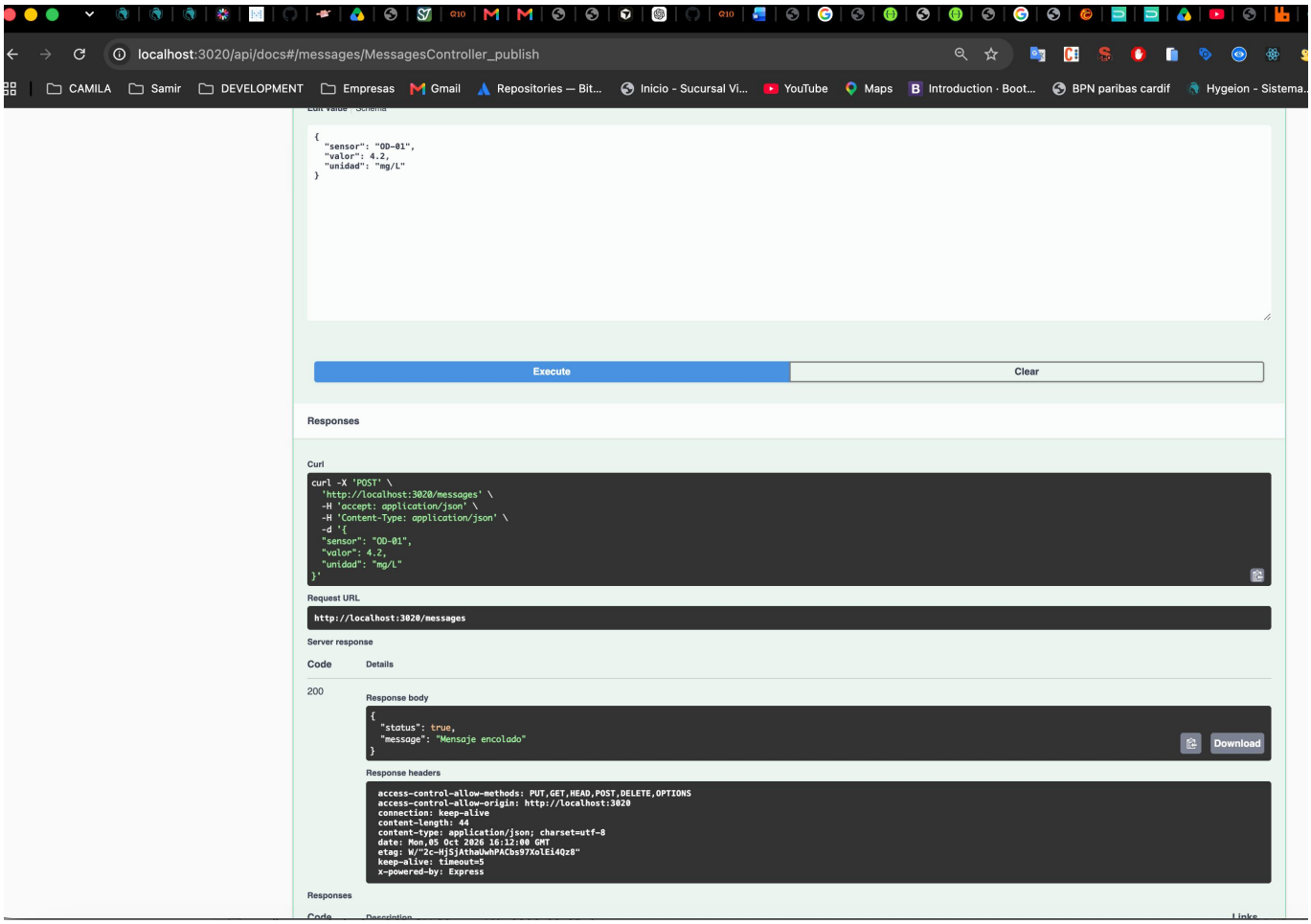
6. Listado posterior a eliminar (GET /clients)



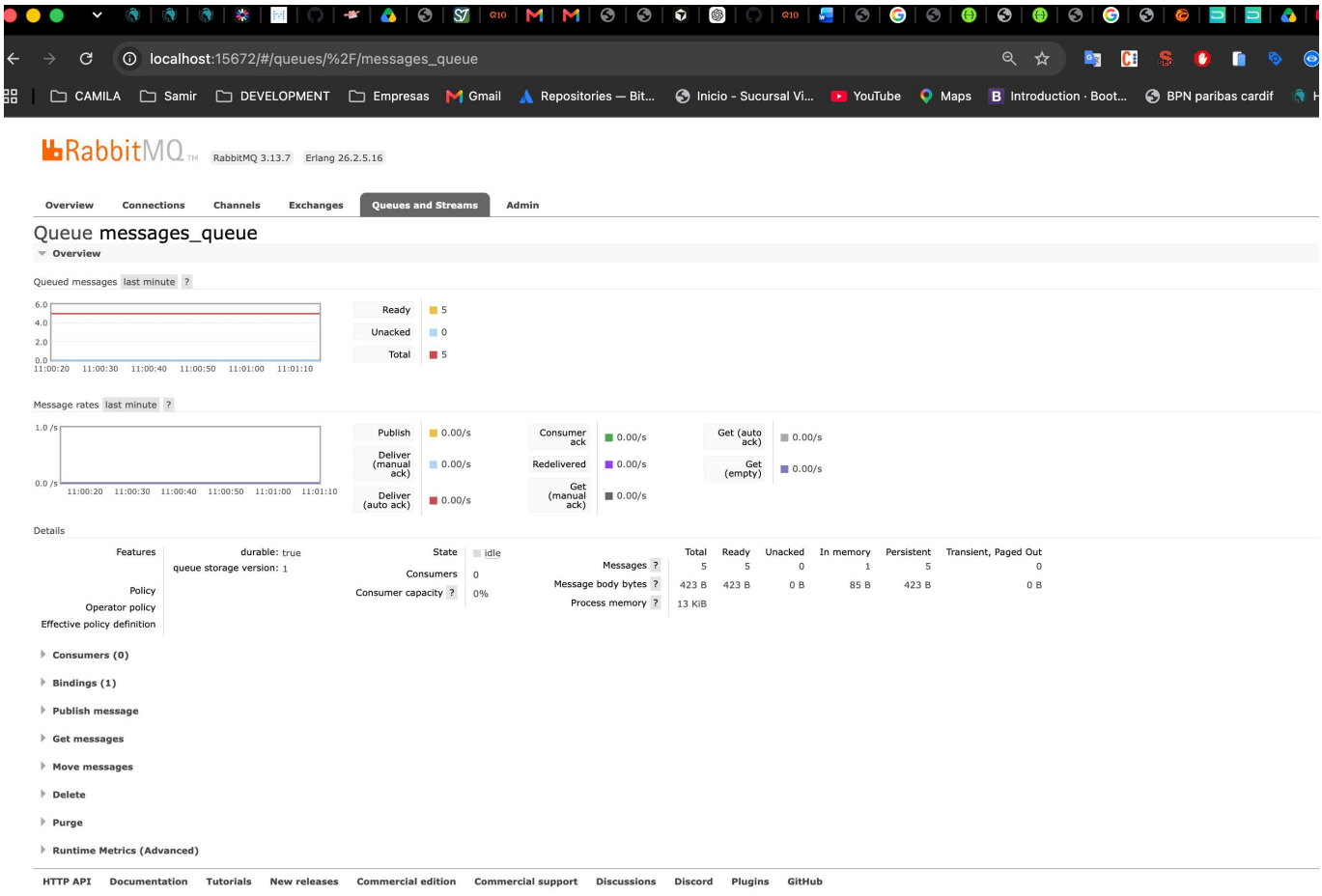
12. Evidencias de RabbitMQ

Capturas del productor y de la consola de administración. La cola es messages_queue.

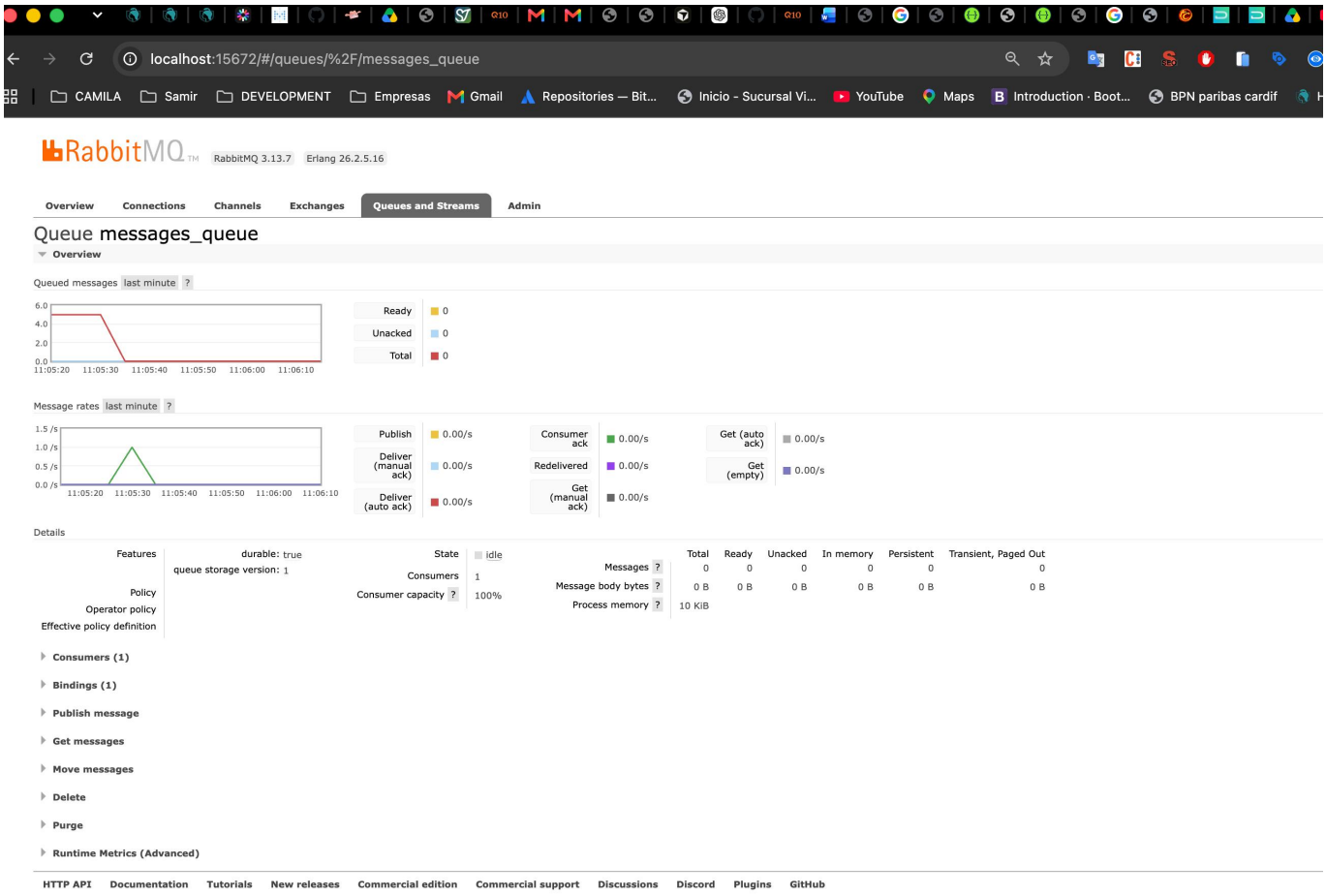
Mensaje válido. POST /messages responde status true y Mensaje encolado.



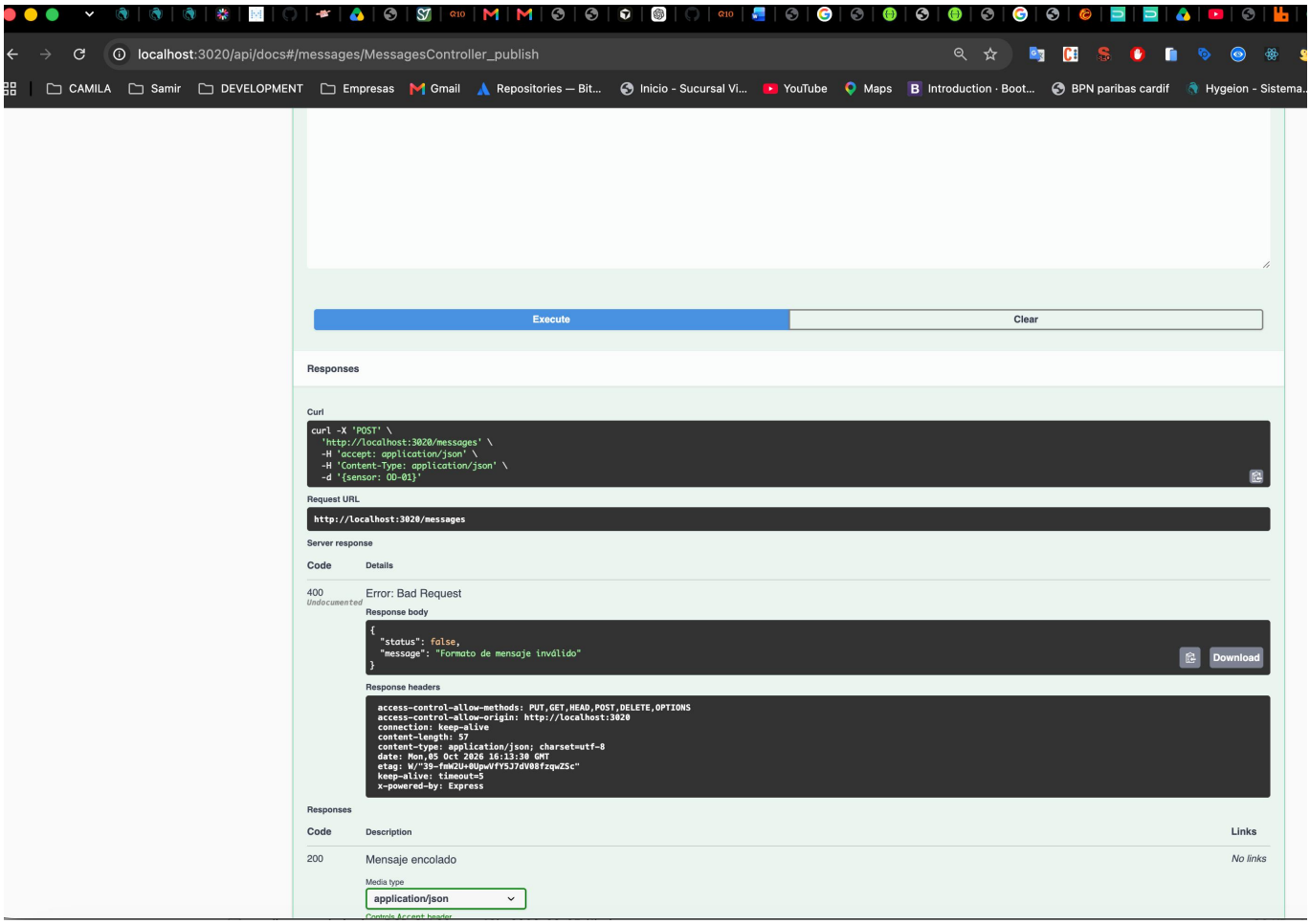
Caso 1, antes. Consumidor detenido: 5 mensajes en Ready. La cola es durable y los 5 mensajes son persistentes.



Caso 1, después. Al levantar el consumidor la cola queda en 0 y aparece 1 consumidor.



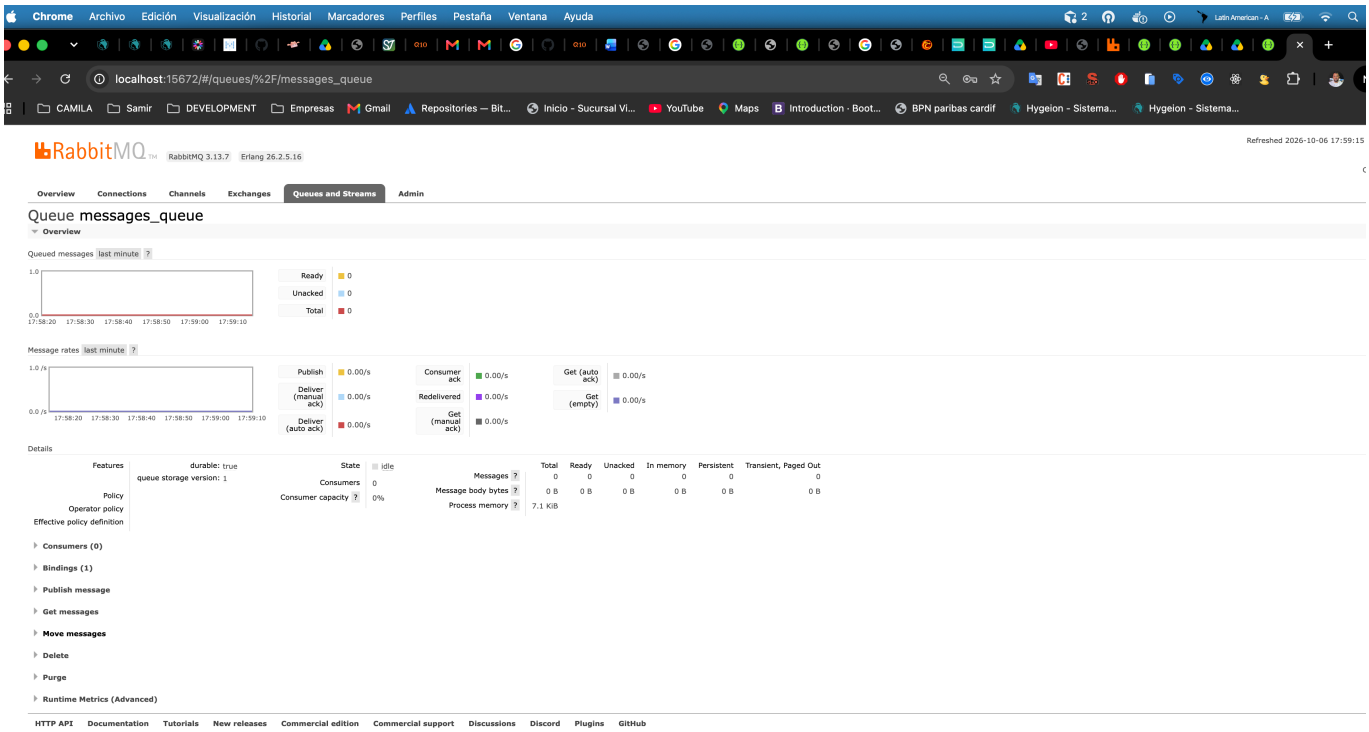
Caso 3. Cuerpo que no es JSON válido: status false y Formato de mensaje inválido. No se publica en la cola.



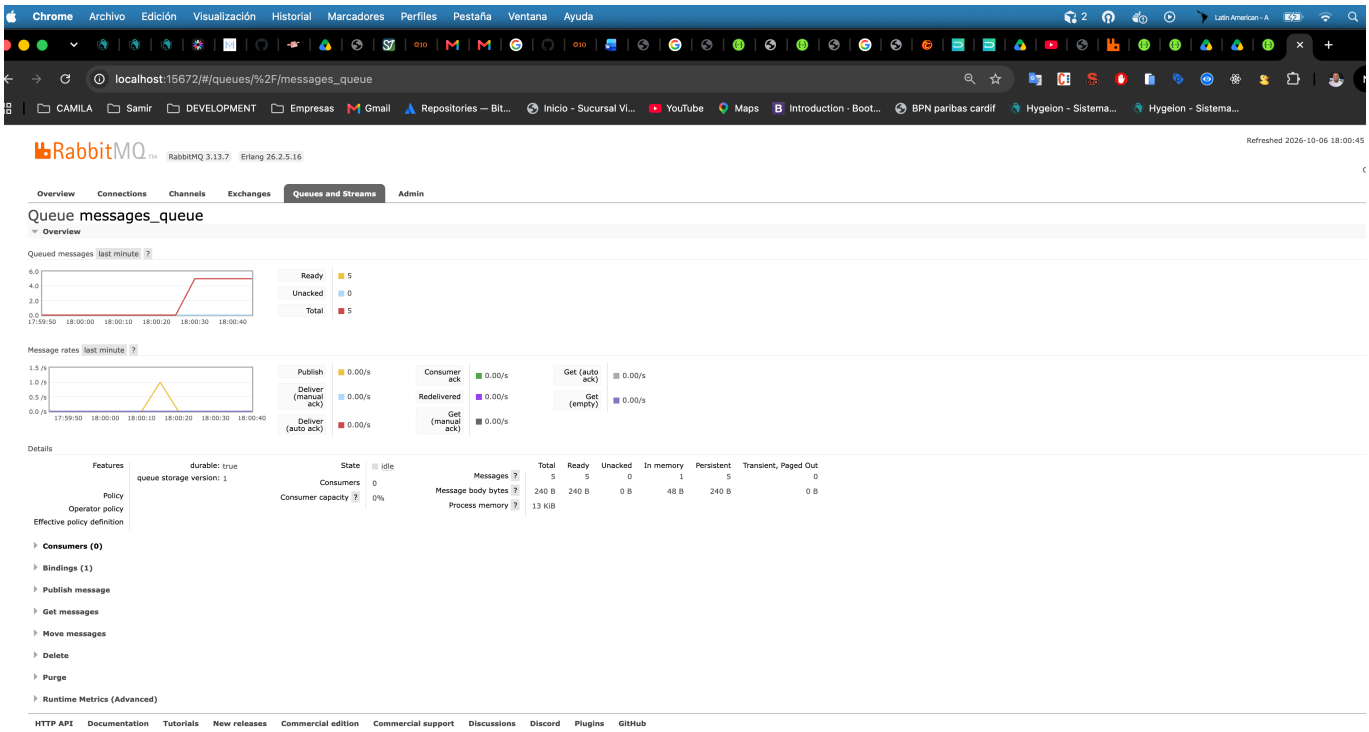
13. Evidencias del taller de pagos

Capturas de la cola, de la tabla pagos y del registro del consumidor para los casos del taller 3.

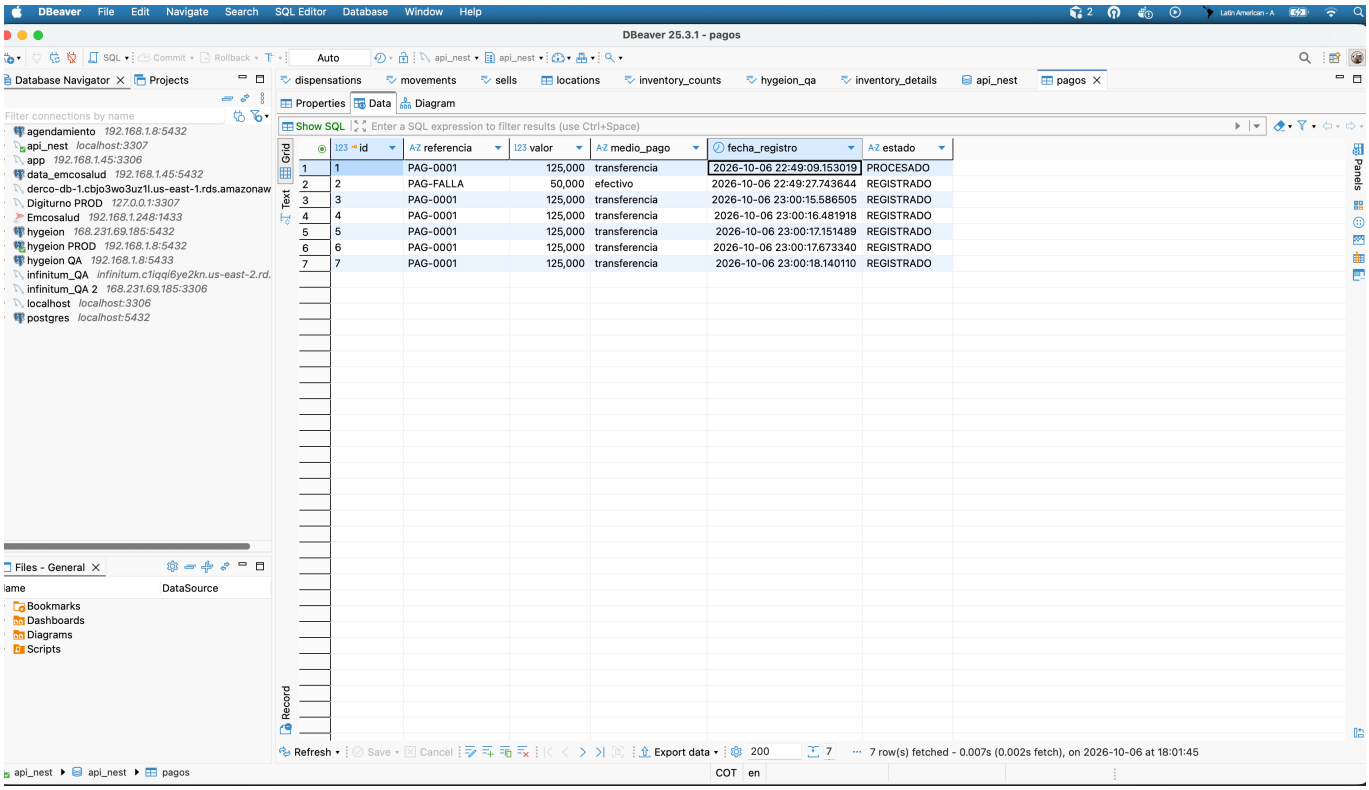
Caso 1, antes. Cola messages_queue vacía, con el consumidor detenido.



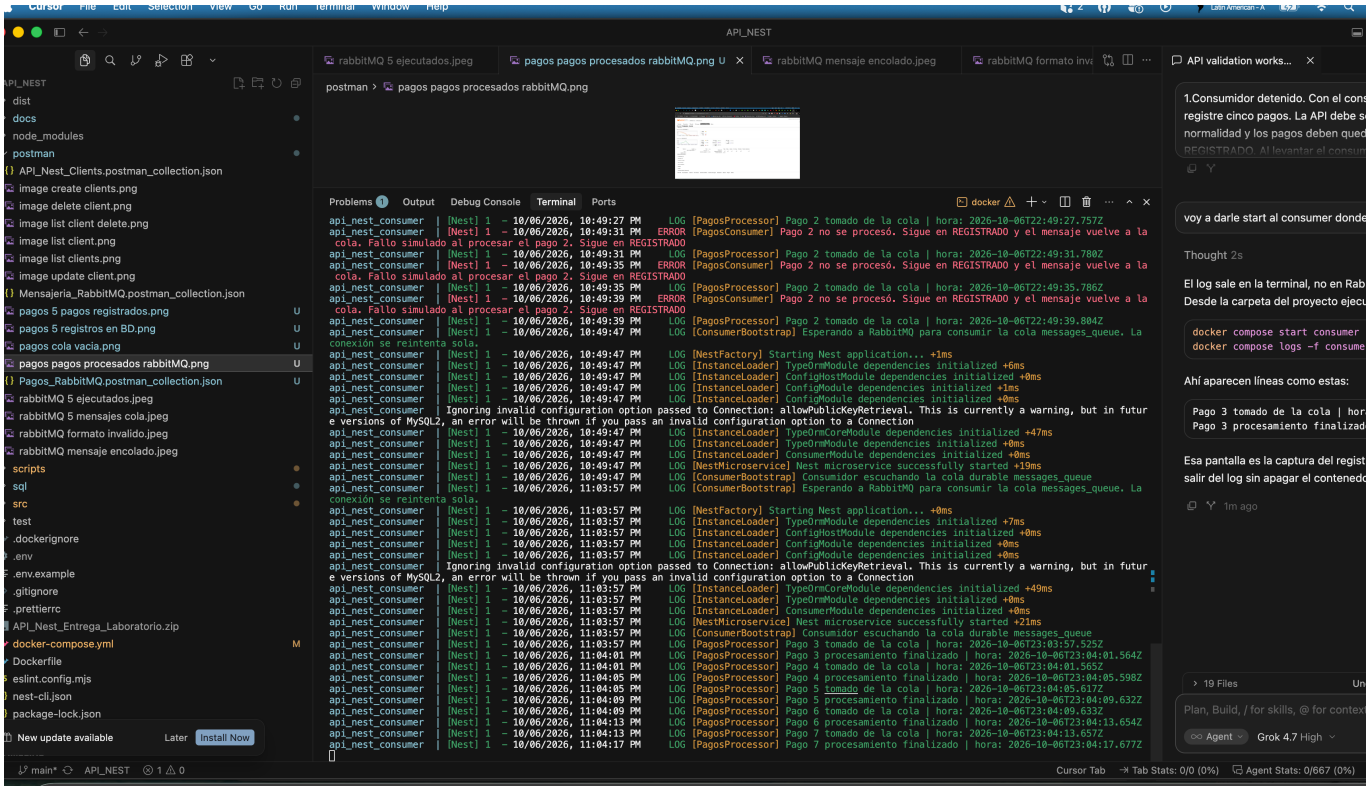
Caso 1, antes. Cinco pagos en Ready, 0 consumidores y mensajes persistentes.



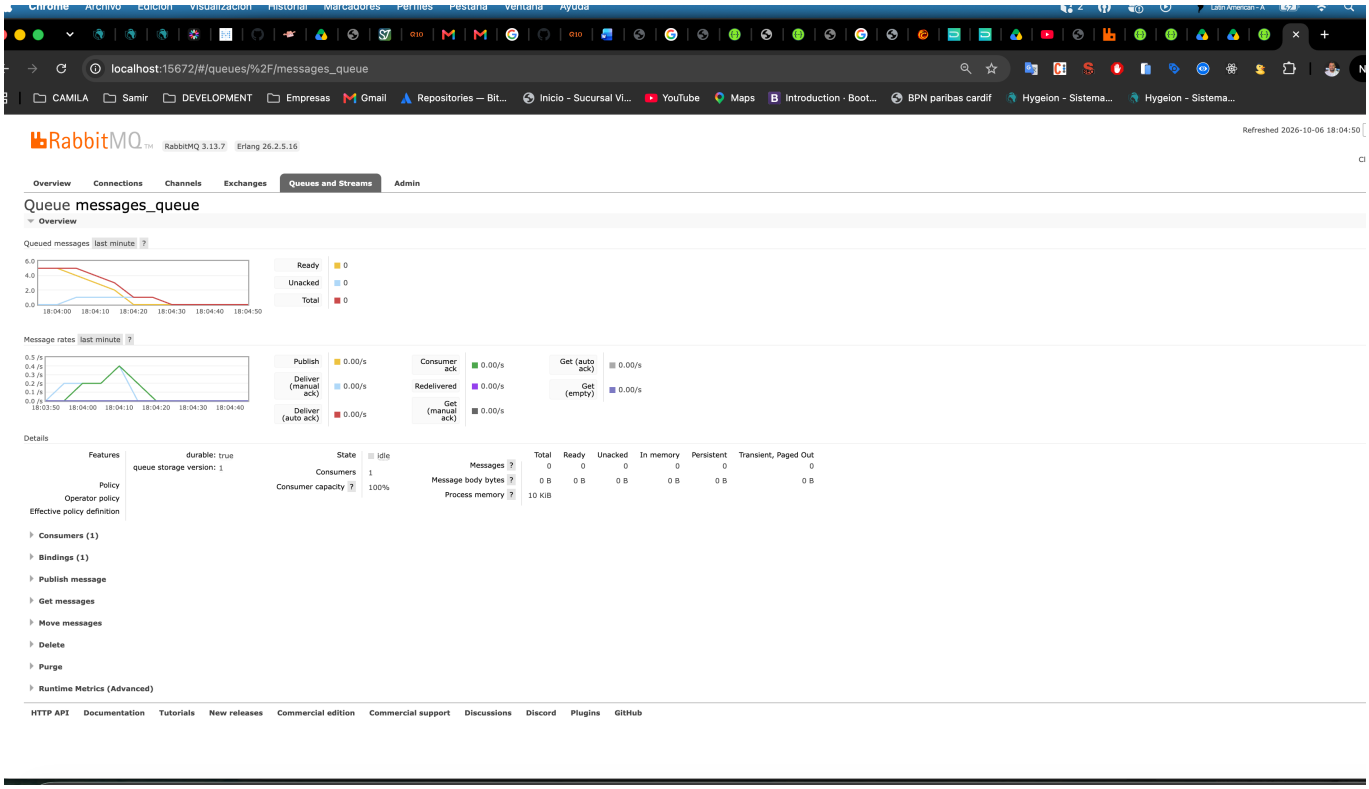
Caso 1, antes. Los pagos 3 a 7 están en REGISTRADO. El pago 2 (PAG-FALLA) también sigue en REGISTRADO.



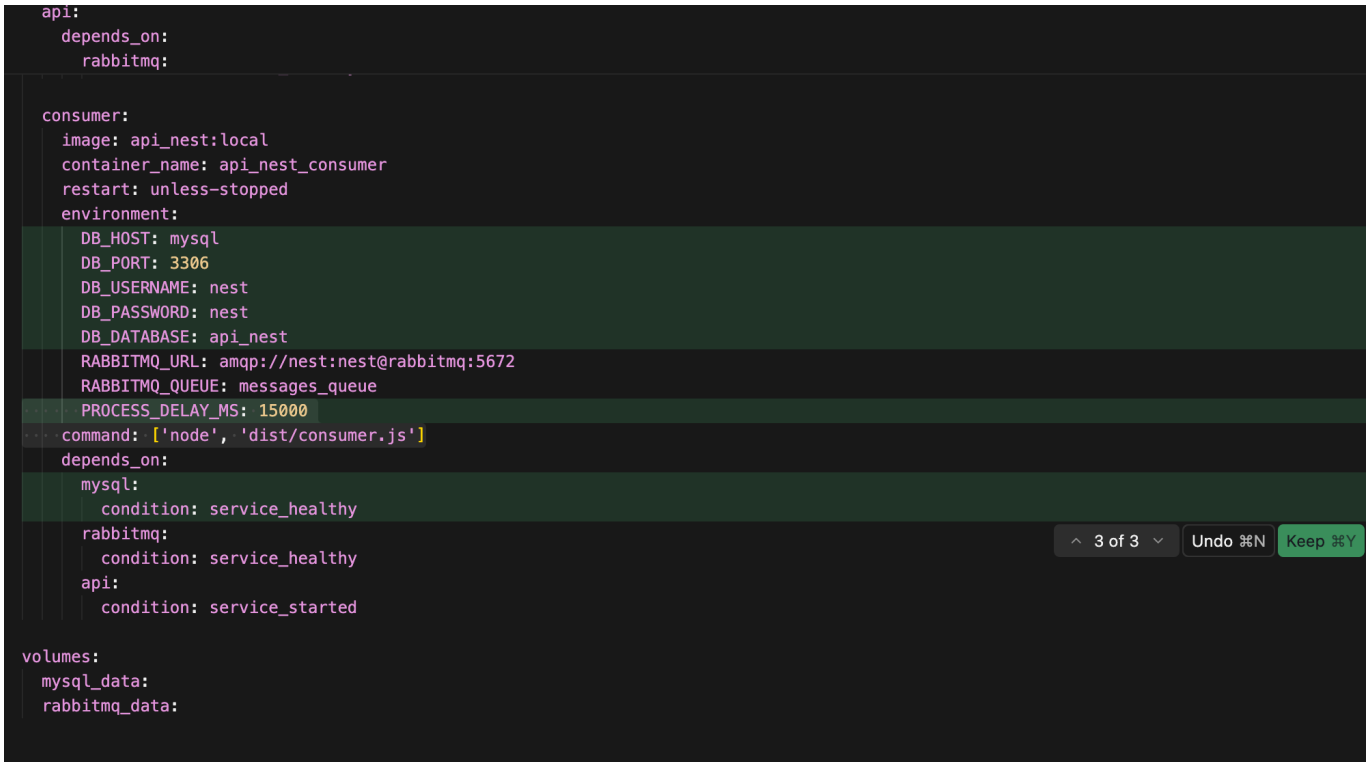
Caso 1, después. El log muestra los pagos 3 a 7 tomados y finalizados. Arriba también se ve el fallo del pago 2, que volvió a la cola.



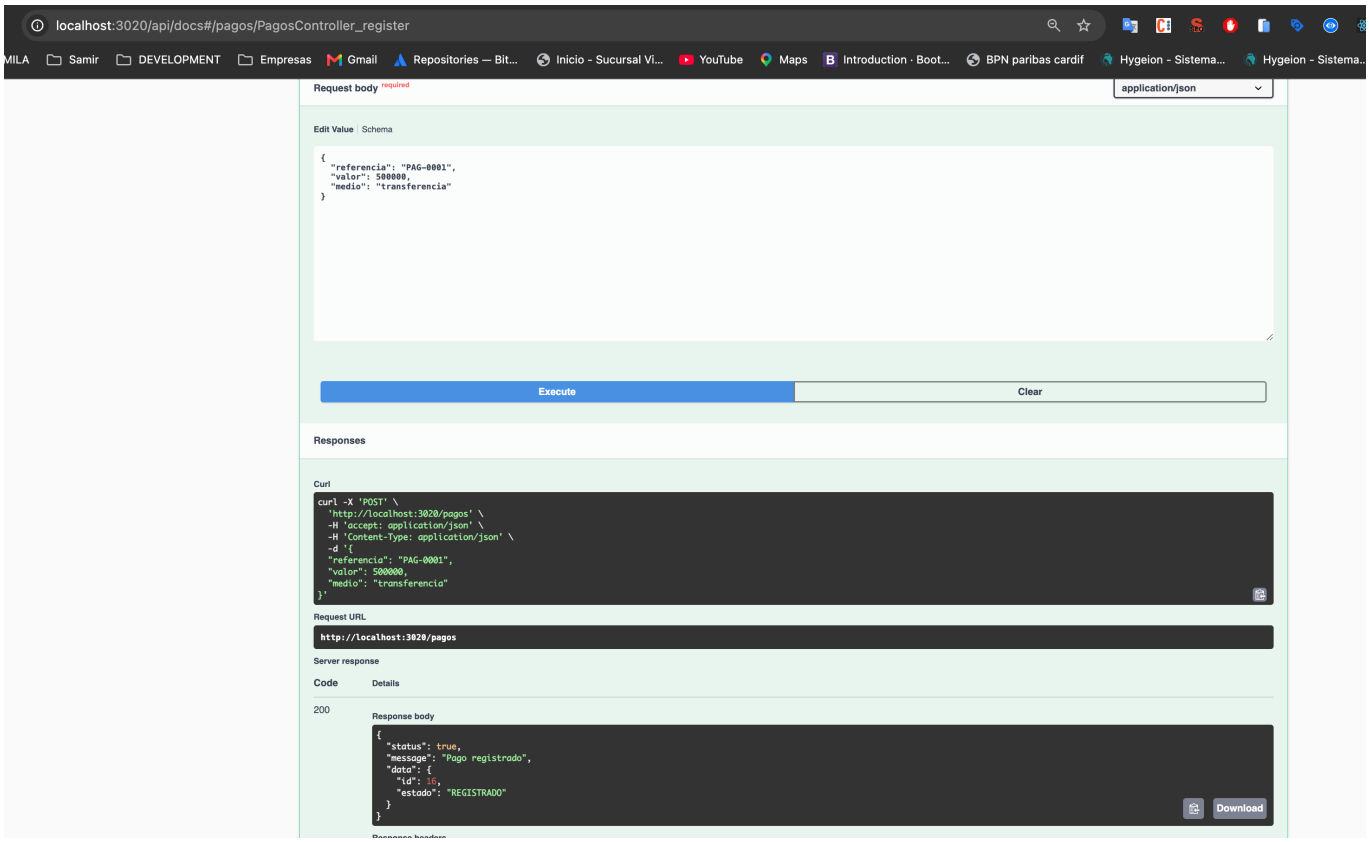
Caso 1, después. La cola queda en 0 y hay 1 consumidor activo.



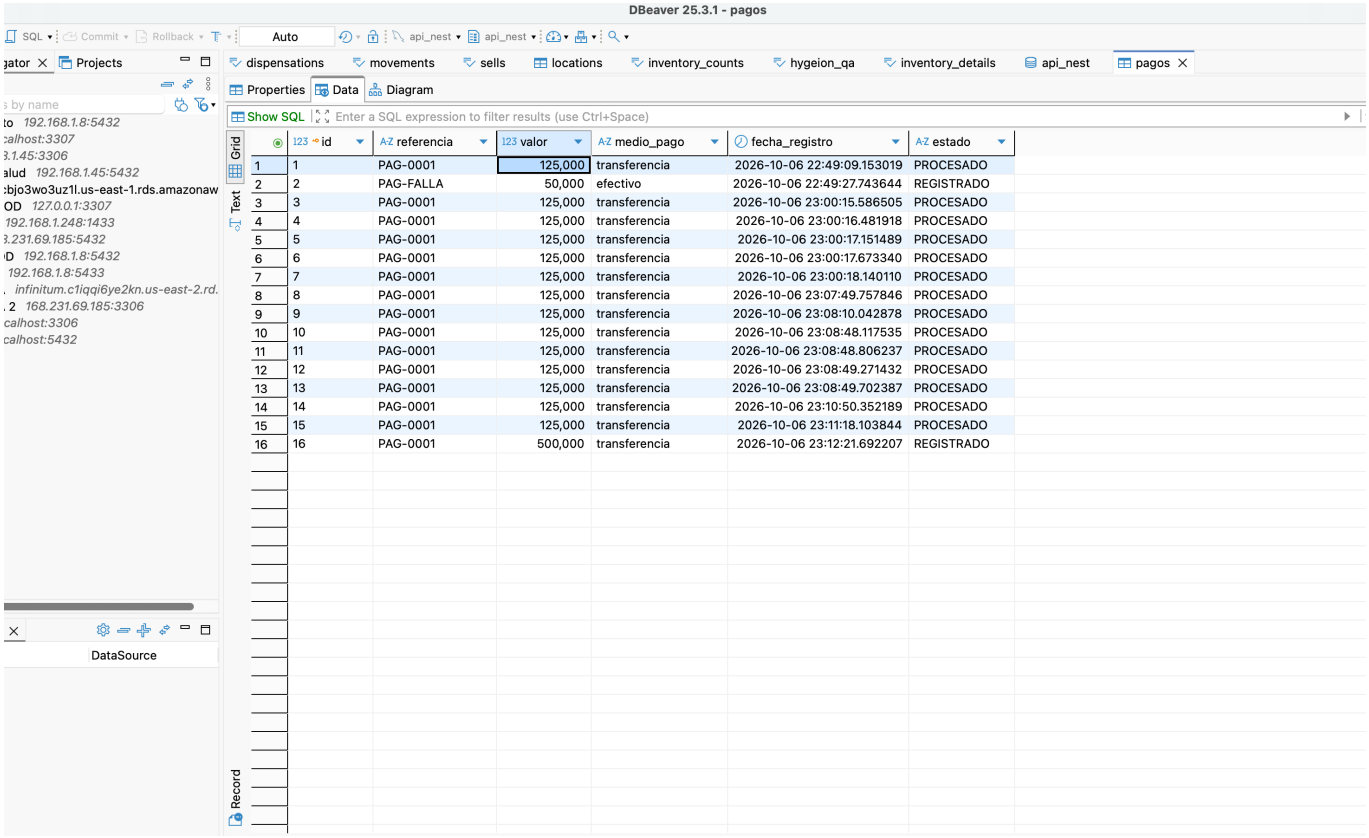
Caso 2. PROCESS_DELAY_MS quedó en 15000.



Caso 2. POST /pagos del pago 16 responde de inmediato con estado REGISTRADO.



Caso 2. Mientras el consumidor espera, el pago 16 sigue en REGISTRADO.

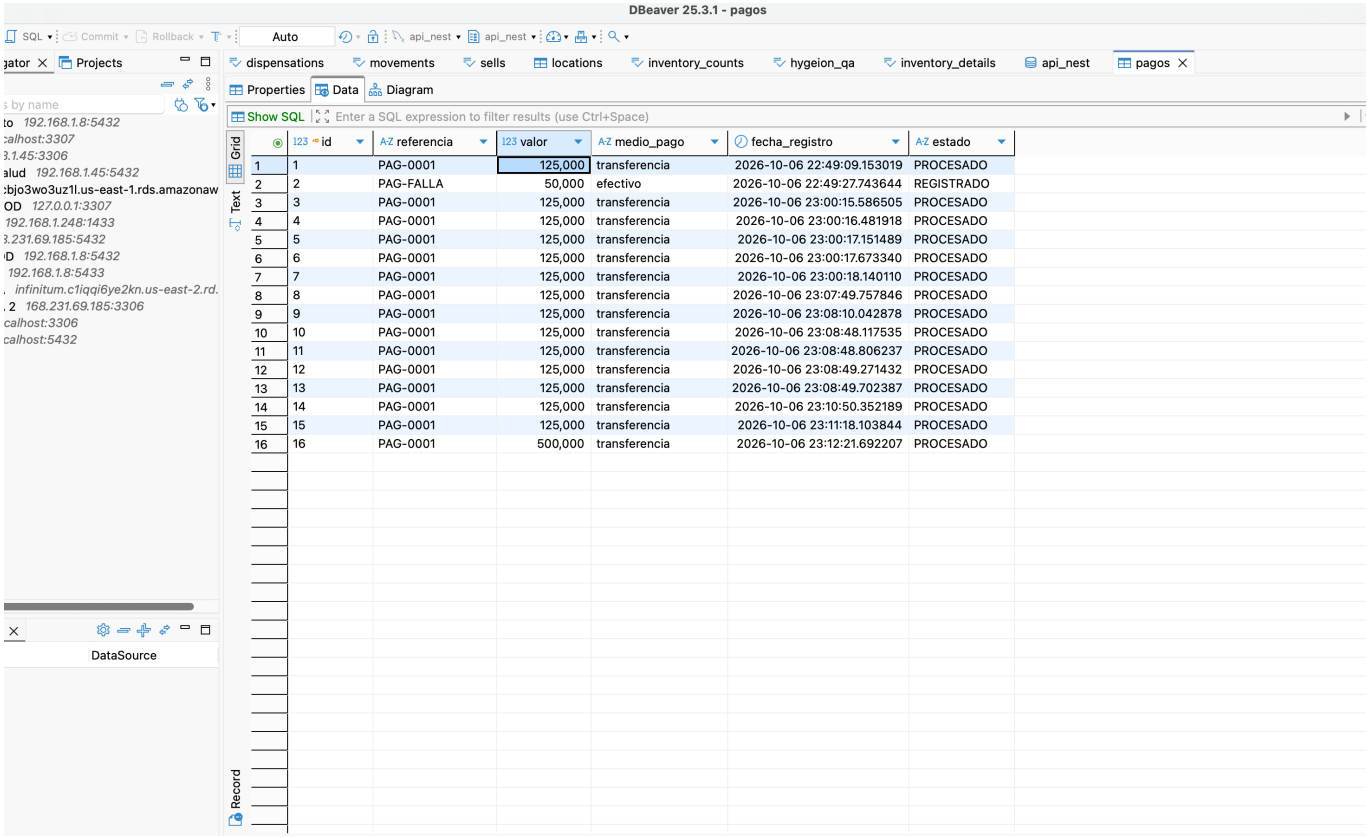


	id	referencia	valor	medio_pago	fecha_registro	estado
1	1	PAG-0001	125,000	transferencia	2026-10-06 22:49:09.153019	PROCESADO
2	2	PAG-FALLA	50,000	efectivo	2026-10-06 22:49:27.743644	REGISTRADO
3	3	PAG-0001	125,000	transferencia	2026-10-06 23:00:15.586505	PROCESADO
4	4	PAG-0001	125,000	transferencia	2026-10-06 23:00:16.481918	PROCESADO
5	5	PAG-0001	125,000	transferencia	2026-10-06 23:00:17.151489	PROCESADO
6	6	PAG-0001	125,000	transferencia	2026-10-06 23:00:17.673340	PROCESADO
7	7	PAG-0001	125,000	transferencia	2026-10-06 23:00:18.140110	PROCESADO
8	8	PAG-0001	125,000	transferencia	2026-10-06 23:07:49.757846	PROCESADO
9	9	PAG-0001	125,000	transferencia	2026-10-06 23:08:10.042878	PROCESADO
10	10	PAG-0001	125,000	transferencia	2026-10-06 23:08:48.117535	PROCESADO
11	11	PAG-0001	125,000	transferencia	2026-10-06 23:08:48.806237	PROCESADO
12	12	PAG-0001	125,000	transferencia	2026-10-06 23:08:49.271432	PROCESADO
13	13	PAG-0001	125,000	transferencia	2026-10-06 23:08:49.702387	PROCESADO
14	14	PAG-0001	125,000	transferencia	2026-10-06 23:10:50.352189	PROCESADO
15	15	PAG-0001	125,000	transferencia	2026-10-06 23:11:18.103844	PROCESADO
16	16	PAG-0001	500,000	transferencia	2026-10-06 23:12:21.692207	REGISTRADO

Caso 2. Log del pago 16: tomado a las 23:12:21.705Z y finalizado a las 23:12:36.759Z (15 segundos).



Caso 2. Tabla de pagos después de registrar el pago 16.



	id	referencia	valor	medio_pago	fecha_registro	estado
1	1	PAG-0001	125,000	transferencia	2026-10-06 22:49:09.153019	PROCESADO
2	2	PAG-FALLA	50,000	efectivo	2026-10-06 22:49:27.743644	REGISTRADO
3	3	PAG-0001	125,000	transferencia	2026-10-06 23:00:15.586505	PROCESADO
4	4	PAG-0001	125,000	transferencia	2026-10-06 23:00:16.481918	PROCESADO
5	5	PAG-0001	125,000	transferencia	2026-10-06 23:00:17.151489	PROCESADO
6	6	PAG-0001	125,000	transferencia	2026-10-06 23:00:17.673340	PROCESADO
7	7	PAG-0001	125,000	transferencia	2026-10-06 23:00:18.140110	PROCESADO
8	8	PAG-0001	125,000	transferencia	2026-10-06 23:07:49.757846	PROCESADO
9	9	PAG-0001	125,000	transferencia	2026-10-06 23:08:10.042878	PROCESADO
10	10	PAG-0001	125,000	transferencia	2026-10-06 23:08:48.117535	PROCESADO
11	11	PAG-0001	125,000	transferencia	2026-10-06 23:08:48.806237	PROCESADO
12	12	PAG-0001	125,000	transferencia	2026-10-06 23:08:49.271432	PROCESADO
13	13	PAG-0001	125,000	transferencia	2026-10-06 23:08:49.702387	PROCESADO
14	14	PAG-0001	125,000	transferencia	2026-10-06 23:10:50.352189	PROCESADO
15	15	PAG-0001	125,000	transferencia	2026-10-06 23:11:18.103844	PROCESADO
16	16	PAG-0001	500,000	transferencia	2026-10-06 23:12:21.692207	PROCESADO